

Session 2 – Programming Language Constructs

Objectives

- Master control flow using conditional statements (`if` , `elif` , `else`) to make decisions in Python programs.
- Understand iteration with `for` and `while` loops for repetitive tasks.
- Learn list comprehensions for concise and efficient list creation.
- Apply these concepts in a hands-on exercise to generate Fibonacci numbers in a Jupyter Notebook.

Key Topics

1. Conditional Statements

- **Purpose:** Allow decision-making based on conditions.
- **Syntax:** Use `if` , `elif` , and `else` with proper indentation (4 spaces in Python).
 - `if` evaluates a condition; if true, executes its block.
 - `elif` checks additional conditions if previous ones fail.
 - `else` runs if no conditions are met.
- **Example:**

```
age = 20
if age >= 18:
    print("You are an adult.")
elif age >= 13:
    print("You are a teenager.")
else:
    print("You are a child.")
```

Output: You are an adult.

- **Key Notes:**

- Conditions must evaluate to a boolean (True or False).
- Use comparison operators (== , != , > , < , >= , <=) and logical operators (and , or , not).
- Example: if age >= 18 and age < 65: print("Adult") .

2. Loops

- **Purpose:** Enable iteration over sequences or until a condition is met.

- **For Loop:**

- Iterates over a sequence (e.g., list, string, or range()).
- Syntax: for variable in sequence:
- Example:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"I like {fruit}")
```

Output:

```
I like apple
I like banana
I like cherry
```

- Using `range(n)` generates numbers from 0 to n-1:

```
for i in range(3):
    print(i)
```

Output: 0 1 2

- **While Loop:**

- Runs as long as a condition is true.
- Syntax: `while condition:`
- Example:

```
count = 0
while count < 3:
    print(count)
    count += 1
```

Output: 0 1 2

- **Key Notes:**

- Ensure `while` loops have a termination condition to avoid infinite loops.
- Use `break` to exit a loop early, `continue` to skip to the next iteration.

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

Output: 0 1 2

3. List Comprehensions

- **Purpose:** Create lists concisely by applying an expression to each item in an iterable, optionally with a condition.
- **Syntax:** [expression for item in iterable if condition]
- **Example:**

```
# Create a list of squares for even numbers
squares = [x**2 for x in range(10) if x % 2 == 0]
print(squares)
```

Output: [0, 4, 16, 36, 64]

- **Equivalent Loop:**

```
squares = []
for x in range(10):
    if x % 2 == 0:
        squares.append(x**2)
```

- **Key Notes:**
 - List comprehensions are more concise and often faster than loops.
 - Can include nested loops: [x*y for x in range(3) for y in range(2)] gives [0, 0, 1, 1, 2, 2] .
 - Avoid overuse for complex logic to maintain readability.

Detailed Explanations with Code Examples

- **Combining Conditionals and Loops:**

```
# Print even numbers with a message
for num in range(10):
    if num % 2 == 0:
        print(f"{num} is even")
    else:
        print(f"{num} is odd")
```

Output:

```
0 is even
1 is odd
2 is even
...
9 is odd
```

- **Nested Loops:**

```
for i in range(3):
    for j in range(2):
        print(f"i={i}, j={j}")
```

Output:

```
i=0, j=0
i=0, j=1
i=1, j=0
...
```

- **List Comprehension with Strings:**

```
words = ["apple", "banana", "cherry"]  
lengths = [len(word) for word in words]  
print(lengths)
```

Output: [5, 6, 6]

Hands-on: Write a Notebook to Generate Fibonacci Numbers

- **Task:** Create a Jupyter Notebook named `fibonacci_generator.ipynb` that generates the first `n` Fibonacci numbers, where each number is the sum of the two preceding ones, starting with 0 and 1 (e.g., 0, 1, 1, 2, 3, 5, 8, ...).
- **Requirements:**
 - Prompt the user for `n` (number of Fibonacci numbers to generate).
 - Use a loop to generate the sequence.
 - Store the sequence in a list and print it.
 - Handle edge cases (e.g., `n <= 0` or `n = 1`).
- **Solution Code:**

```

# Fibonacci Generator in Jupyter Notebook
print("Fibonacci Number Generator")

# Get user input
try:
    n = int(input("Enter the number of Fibonacci numbers to generate: "))
    if n <= 0:
        print("Please enter a positive number.")
    else:
        # Initialize Fibonacci list
        fib = [0, 1] if n > 1 else [0] if n == 1 else []

        # Generate Fibonacci numbers
        while len(fib) < n:
            fib.append(fib[-1] + fib[-2])

        # Output the result
        print(f"The first {n} Fibonacci numbers are: {fib[:n]}")
except ValueError:
    print("Invalid input. Please enter a valid integer.")

```

- **Explanation:**

- The program starts with `[0, 1]` for `n > 1`, or `[0]` for `n = 1`.
- The `while` loop appends the sum of the last two numbers until the list has `n` elements.
- The slice `fib[:n]` ensures exactly `n` numbers are shown.
- Error handling with `try/except` catches invalid inputs (e.g., non-integers).

- **Example Output:**

Fibonacci Number Generator

Enter the number of Fibonacci numbers to generate: 7

The first 7 Fibonacci numbers are: [0, 1, 1, 2, 3, 5, 8]

- **Alternative with List Comprehension** (less readable, for reference):

```
n = 7
fib = [0, 1]
[fib.append(fib[-1] + fib[-2]) for _ in range(n-2)]
print(fib[:n])
```

- Note: Loops are preferred here for clarity and to avoid side effects in comprehensions.

Additional Notes

- **Best Practices:**

- Always indent consistently (use 4 spaces, not tabs).
- Use meaningful variable names (e.g., `fib` instead of `x`).
- Comment complex logic for clarity.
- Test edge cases: What happens if `n = 0` or input is invalid?

- **Common Pitfalls:**

- Forgetting to update loop variables in `while` loops (causes infinite loops).
- Incorrect indentation leading to syntax errors.
- Overcomplicating list comprehensions when a loop is clearer.

- **Jupyter Notebook Tips:**

- Use markdown cells for explanations and code cells for executable code.
- Run cells with Shift+Enter; restart the kernel if variables get messy.
- Save frequently (`ctrl+s`) to avoid losing work.

Practice Exercises

1. Modify the Fibonacci program to use a `for` loop instead of `while` .
2. Write a list comprehension to generate a list of odd numbers from 1 to 20.
3. Create a program that prints a multiplication table (1 to 10) using nested loops.
4. Add error handling to the Fibonacci program to handle negative inputs gracefully.

Resources

- Python Documentation: [Control Flow](#)
- Jupyter Notebook: [Official Guide](#)
- Online Practice: [w3schools Python](#)